

Sprawozdanie 2

Wybrane Zagadnienia Algebry 2026 · Lab. 4

Imię i nazwisko: Jędrzej Sajnog

Nr indeksu: 279701

Lab. 4a — Wielomian wielu zmiennych

Wielomiany wielu zmiennych są reprezentowane jako Python dictionary, gdzie kluczem jest krotka wykładników, a wartością współczynnik:

```
# f(x,y,z) = 4x^3*y^2 - 2xy^2*z + z^4 + 1
f = { (3,2,0): 4, (1,2,1): -2, (0,0,4): 1, (0,0,0): 1 }
```

Zaimplementowane zostały również standardowe operacje takie jak dodawanie, odejmowanie, mnożenie itp.

Lab. 4b — Porządki na jednomianach

Zaimplementowane zostały 3 porządki:

- **Lex** — porównujemy wykładniki od lewej, jak słowa

w słowniku.

- **Permutowany Lex** — to Lex z inną kolejnością zmiennych,
- **GradedLex (grlex)** — najpierw porównuje łączny stopień, przy remisie używa Lex.

```
def lex_cmp(exp1, exp2, perm=None):
    if perm is not None:
        exp1 = tuple(exp1[i] for i in perm)
        exp2 = tuple(exp2[i] for i in perm)
    for a, b in zip(exp1, exp2):
        if a != b: return 1 if a > b else -1
    return 0

def grlex_cmp(exp1, exp2):
    d1, d2 = sum(exp1), sum(exp2)
    if d1 != d2: return 1 if d1 > d2 else -1
    return lex_cmp(exp1, exp2)
```

Lab. 4c — Algorytm PolynomialReduce

Algorytm dzieli wielomian f przez listę $G = (g_1, \dots, g_n)$, wyznaczając rozkład:

$$f = \alpha_1 g_1 + \alpha_2 g_2 + \dots + \alpha_n g_n + r$$

gdzie reszta r spełnia: żaden wyraz r nie jest podzielny przez $LT(g_i)$ dla żadnego i .

```

def polynomial_reduce(f, G, cmp_fn):
    alphas = [{ } for _ in range(len(G))]
    r = { }
    p = dict(f)

    while p:
        lt_exp, lt_coeff = leading_term(p, cmp_fn)
        divided = False

        for i, g in enumerate(G):
            g_lt_exp, g_lt_coeff = leading_term(g, cmp_fn)
            if divides(g_lt_exp, lt_exp):
                q_exp, q_coeff = monomial_div(
                    lt_exp, lt_coeff, g_lt_exp, g_lt_coeff)
                alphas[i][q_exp] = alphas[i].get(q_exp, 0) + q_coeff
                alphas[i] = poly_clean(alphas[i])
                p = poly_sub(p, poly_mul_monomial(g, q_exp, q_coeff))
                divided = True
                break

        if not divided:
            r[lt_exp] = r.get(lt_exp, 0) + lt_coeff
            del p[lt_exp]

    return alphas, poly_clean(r)

```

Jest to standardowy algorytm dzielenia „pisemnego”.

1. wczytujemy wartości: $p = f$, $\alpha_i = 0$, $r = 0$
2. wybieramy największy wyraz p
3. Dla każdego $g_i \in G$ po kolei sprawdzamy czy $LT(g_i)$ dzieli $LT(p)$, jeśli tak to wyznaczamy odpowiedni mnożnik q
4. Dodajemy q do odpowiedniego α_i i odejmujemy od p
5. Jeśli żaden g_i nie spełnia wymogu to przenosimy to do reszty r i usuwamy z p
6. Kończymy gdy $p = 0$

Lab. 4d – Ćwiczenie 37

Dane: $f = x^3 - x^2y - x^2z$, $g_1 = x^2y - z$, $g_2 = xy - 1$, porządek GradedLex. Wyniki dzielenia:

Kolejność	α_1	α_2	Reszta r
(g_1, g_2)	-1	0	$x^3 - x^2z - z$
(g_2, g_1)	$-x$	0	$x^3 - x^2z - x$

Różnice wynikają z tego że w kroku 2 obu redukcji $LT(p) = -x^2y$. Algorytm próbuje dzielników po kolei:

- w (g_1, g_2) : $LT(g_1) = x^2y$ dzieli $-x^2y$ do -1;
- w (g_2, g_1) : $LT(g_2) = xy$ też dzieli $-x^2y$ do $-x$.

Ta różnica wpływa na ostateczne wartości.

Czy $r_1 - r_2 \in \langle g_1, g_2 \rangle$? $r_1 - r_2 = x - z$. Da się to zapisać jako kombinację g_1 i g_2 : $x - z = (x^2y - z) - x(xy - 1) = g_1 - xg_2$. Zatem różnica reszt należy do ideału generowanego przez g_1 i g_2 .

Lab. 4e – Wielomian własny

$$h(x, y, z) = x^2y^7 - y^9z^7 + z$$

Wybrane 3 porządki i ciąg G :

Wybrane wielomiany:

- $g_1 = x^2y^7 - z$
- $g_2 = y^9z^7 - z$
- $g_3 = z - x$

Wybrane zostały 3 porządki - permutowany LEX:

- $x > y > z$
- $z > x > y$
- $y > z > x$

Porządek	Reszta r
$x > y > z$	z
$y > z > x$	x
$z > x > y$	x^2y^7

Dlaczego wyniki są różne?

Wielomiany g_i zostały wybrane w taki sposób by w zależności od tego który współczynnik jest uznawany za największy „usunąć” inny monom z funkcji h w trakcie wykonywania algorytmu. W ten sposób możemy sprawić że w zależności od wybranego porządku inna funkcja była wykorzystana do dzielenia, prowadząc do innej reszty.

Lab. 5 Godło Japonii

Do narysowania wykresów wykorzystałem matplotlib. Wykorzystany wzór (we współrzędnych biegunowych):

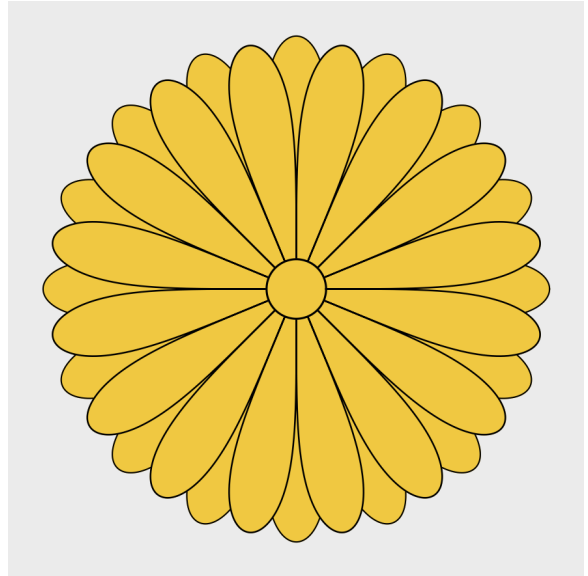
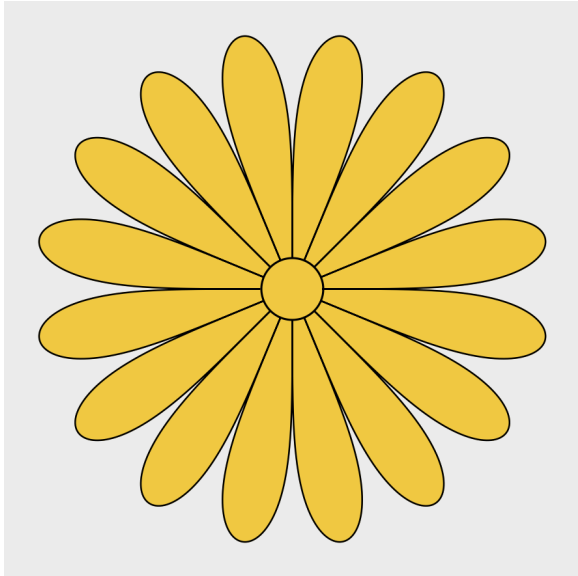
$$r = |\sin(8\theta)|^{\frac{1}{4}}$$

gdzie $\theta \in [0, 2\pi)$. Wzór jest inspirowany równaniem Quadrifolium z paroma zmianami:

- wartość bezwzględna - uniknięcie problemów z potęgowaniem
- 8θ - by uzyskać 16 płatków
- $^{\frac{1}{4}}$ - by przeskalować płatki - by były „grubsze”

Dodatkowo na środku narysowane jest koło.

W ten sposób otrzymujemy godło podobne do tego, które znajduje się na paszporcie japońskim. W celu dodania drugiego rzędu płatków starczy narysować najpierw w tle ten sam obraz ale przesunięty z wartością θ przesuniętą o jakiś offset. Ja eksperymentalnie wyznaczyłem jako offset $\frac{\pi}{16}$



Krzywe Algebraiczne: Bierzemy wzór $r = |\sin(8\theta)|^{\frac{1}{4}}$ po podniesieniu do 8 potęgi otrzymujemy $r^8 = \sin^2(8\theta)$. Mamy $(x + yi)^8 = r^8(\cos(8\theta) + i \sin(8\theta))$, zatem $Im((x + yi)^8) = r^8 \sin(8\theta)$, podnosimy do kwadratu i dostajemy $r^{16} \sin^2(8\theta) = Im((x + yi)^8)^2$, $r^2 = x^2 + y^2$ i $\sin^2(8\theta) = r^8$ zatem $r^{24} = (x^2 + y^2)^{12}$. Zatem ostatecznie równanie krzywej ma postać:

$$(x^2 + y^2)^{12} = Im((x + yi)^8)^2$$

lub jako już wyliczony wielomian:

$$(x^2 + y^2)^{12} - (8xy(x - y)(x + y)(x^2 - 2xy - y^2)(x^2 + 2xy - y^2))^2 = 0$$

By dodać okrąg po środku wystarczy wziąć równanie okręgu:

$$x^2 + y^2 = c^2$$

I te równanie okręgu pomnożyć razy równanie „płatków”. Wtedy otrzymamy pełny wykres.

Zatem finalnym równaniem jest:

$$\left[(x^2 + y^2)^{12} - (8xy(x - y)(x + y)(x^2 - 2xy - y^2)(x^2 + 2xy - y^2))^2 \right] (x^2 + y^2 - c^2) = 0$$